



CS663

HAND GESTURE RECOGNITION

AYUSH VIJAY AWATADE

NIKHIL KHANDELWAL

Brief Overview Of The Project

02

This project aims to learn American Sign Language (ASL) hand gestures using a computer vision model that detects each gesture.

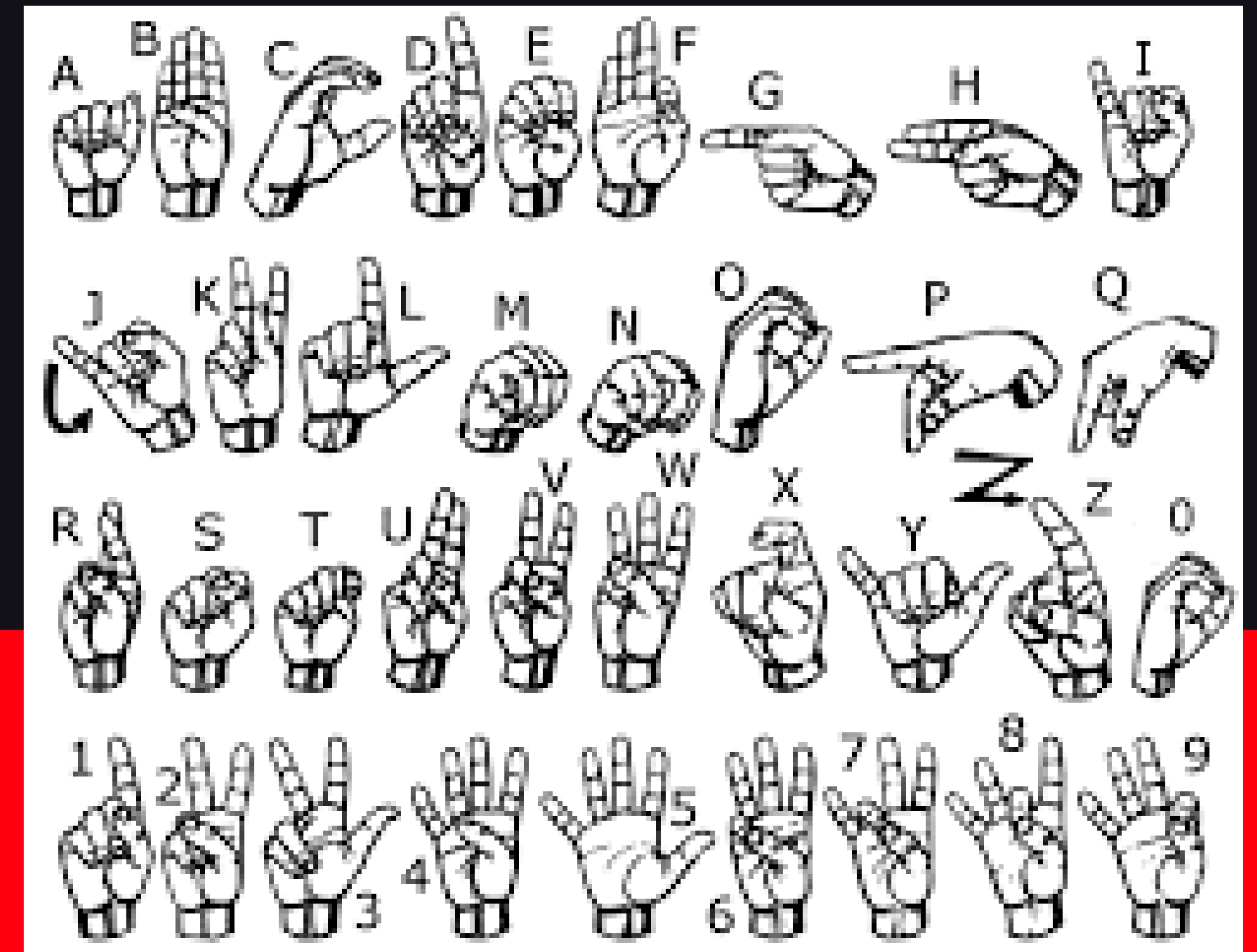


Table of Contents

- 1) Data Extraction and Exploration
- 2) Data Preprocessing and Preparation
- 3) Model Training
- 4) Model Evaluation

The data is taken from kaggle(open-source), for the alphabets ranging from a-z and numbers 0-9, and is analyzed further.

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("ayuraj/asl-dataset")

print("Path to dataset files:", path)

Downloading from https://www.kaggle.com/api/v1/datasets/download/ayuraj/asl-dataset?dataset\_version\_number=1..
100%|██████████| 56.9M/56.9M [00:00<00:00, 209MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/ayuraj/asl-dataset/versions/1

for label in sorted(os.listdir(path)):
    folder = os.path.join(path, label)
    if os.path.isdir(folder):
        print(label, "->", len(os.listdir(folder)), "directories")

asl_dataset -> 37 directories
```

04

Data Extraction

Data Exploration

```
dataset_path = "/root/.cache/kagglehub/datasets/ayuraj/asl-dataset/versions/1/asl_dataset/asl_dataset"

print("Dataset path:", dataset_path)
print("\nFolders inside dataset:")
for item in sorted(os.listdir(dataset_path)):
    print("  ", item)
```

```
Dataset path: /root/.cache/kagglehub/datasets/ayuraj/asl-dataset/versions/1/asl_dataset/asl_dataset
```

Folders inside dataset:

- 0
- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- a
- b
- c
- d
- e
- f
- g
- h
- i
- j
- k
- l
- m
- n
- o
- p
- q
- r
- s
- t
- u
- v
- w
- x
- y
- z

```
print("\nImages per class:\n")
class_counts = {}

for label in sorted(os.listdir(dataset_path)):
    class_dir = os.path.join(dataset_path, label)
    if os.path.isdir(class_dir):
        count = len([f for f in os.listdir(class_dir) if f.lower().endswith((".jpg", ".jpeg", ".png"))])
        class_counts[label] = count
        print(f"{label}: {count} images")

print("\nTotal classes:", len(class_counts))
print("Total images:", sum(class_counts.values()))
```

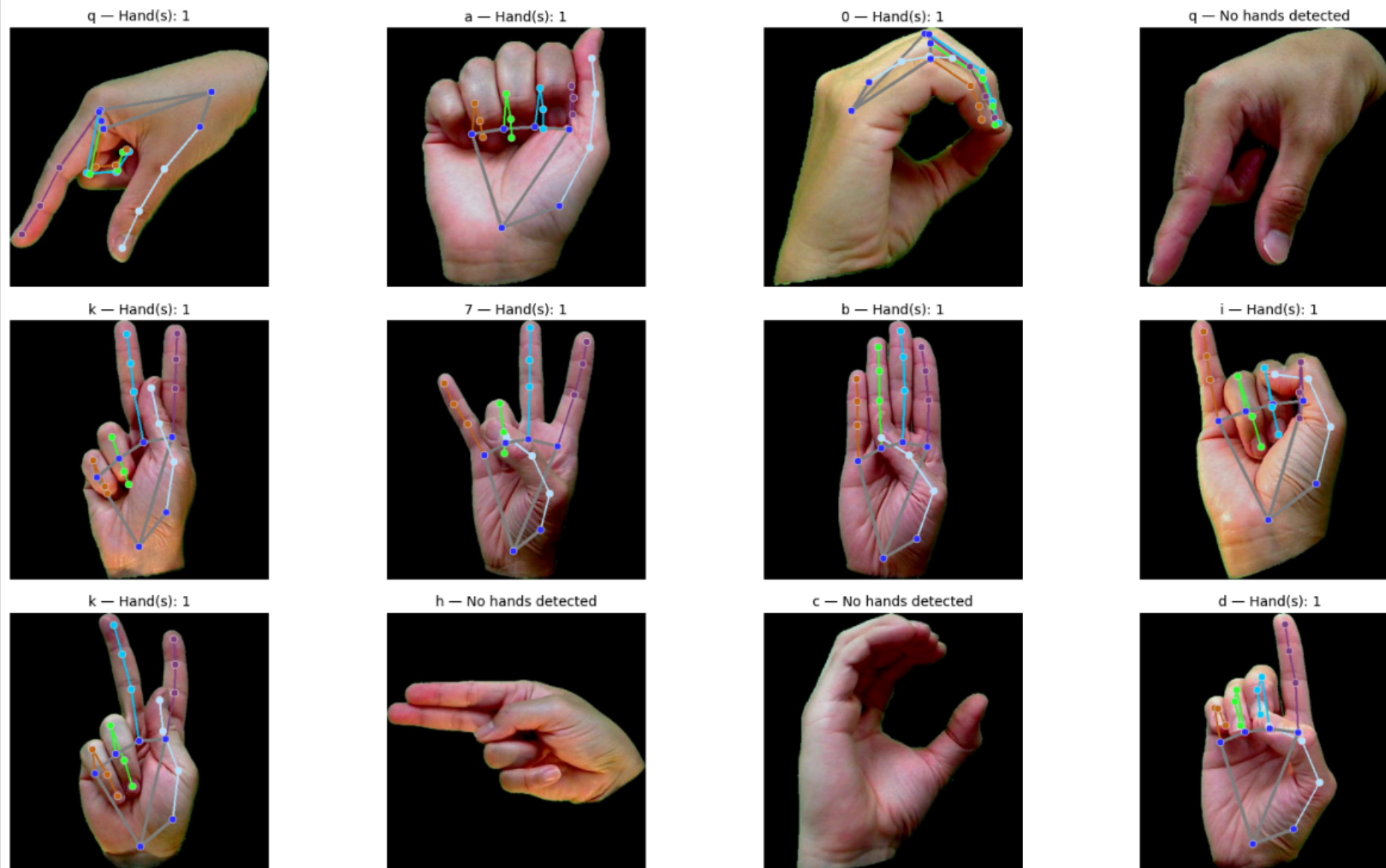
Images per class:

0: 70 images
1: 70 images
2: 70 images
3: 70 images
4: 70 images
5: 70 images
6: 70 images
7: 70 images
8: 70 images
9: 70 images
a: 70 images
b: 70 images
c: 70 images
d: 70 images
e: 70 images
f: 70 images
g: 70 images
h: 70 images
i: 70 images
j: 70 images
k: 70 images
l: 70 images
m: 70 images
n: 70 images
o: 70 images
p: 70 images
q: 70 images
r: 70 images
s: 70 images
t: 65 images
u: 70 images
v: 70 images
w: 70 images
x: 70 images
y: 70 images
z: 70 images

Total classes: 36
Total images: 2515

Data Exploration

Found 2515 images across 36 classes.



```
import random
import cv2
import mediapipe as mp
import matplotlib.pyplot as plt

all_image_paths = []
for label in sorted(os.listdir(dataset_path)):
    class_dir = os.path.join(dataset_path, label)
    if not os.path.isdir(class_dir):
        continue
    for fname in os.listdir(class_dir):
        if fname.lower().endswith((".png", ".jpg", ".jpeg")):
            all_image_paths.append((label, os.path.join(class_dir, fname)))

print("Found", len(all_image_paths), "images across", len(os.listdir(dataset_path)), "classes.")

sampled = random.sample(all_image_paths, min(12, len(all_image_paths))) #Plotting images to see the dataset

# MediaPipe
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
mp_styles = mp.solutions.drawing_styles

# Use context manager so resources are freed automatically
with mp_hands.Hands(static_image_mode=True, min_detection_confidence=0.25) as hands:
    plt.figure(figsize=(16, 9))
    cols = 4
    rows = (len(sampled) + cols - 1) // cols

    for idx, (label, img_path) in enumerate(sampled):
        img = cv2.imread(img_path)
        if img is None:
            print("Could not read", img_path)
            continue

        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        # run MediaPipe
        results = hands.process(img_rgb)

        # create a copy for drawing
        annotated = img_rgb.copy()
        if results.multi_hand_landmarks:
            for hand_landmarks in results.multi_hand_landmarks:
                mp_drawing.draw_landmarks(
                    annotated,
                    hand_landmarks,
                    mp_hands.HAND_CONNECTIONS,
                    mp_styles.get_default_hand_landmarks_style(),
                    mp_styles.get_default_hand_connections_style()
                )
            status = f"{label} — Hand(s): {len(results.multi_hand_landmarks)}"
        else:
            status = f"{label} — No hands detected"

        # plot
        plt_idx = idx + 1
        ax = plt.subplot(rows, cols, plt_idx)
        ax.imshow(annotated)
        ax.set_title(status, fontsize=10)
        ax.axis("off")

    plt.tight_layout()
    plt.show()
```

Data Exploration

```
import math
classes = sorted([d for d in os.listdir(dataset_path) if os.path.isdir(os.path.join(dataset_path, d))])

# prepare report storage
report_rows = []
overall_total = 0
overall_detected = 0
overall_not_detected = 0
overall_unreadable = 0

# run MediaPipe Hands in a with-context so it frees resources afterward
with mp_hands.Hands(static_image_mode=True, min_detection_confidence=0.25) as hands:
    for cls in classes:
        class_dir = os.path.join(dataset_path, cls)
        # count metrics for this class
        total = 0
        detected = 0
        not_detected = 0
        unreadable = 0

        # list files (only common image extensions)
        files = sorted([f for f in os.listdir(class_dir) if f.lower().endswith((".png", ".jpg", ".jpeg"))])

        for fname in files:
            total += 1
            img_path = os.path.join(class_dir, fname)
            try:
                img = cv2.imread(img_path)
                if img is None:
                    unreadable += 1
                    continue

                img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                results = hands.process(img_rgb)

                if results.multi_hand_landmarks:
                    detected += 1
                else:
                    not_detected += 1

            except Exception as e:
                unreadable += 1

        # compute detection rate (consider only readable images)
        readable = total - unreadable
        detection_rate = (detected / readable * 100) if readable > 0 else float('nan')
```

```
report_rows.append({
    "class": cls,
    "total_images": total,
    "unreadable_files": unreadable,
    "readable_images": readable,
    "images_with_hands": detected,
    "images_with_no_hands": not_detected,
    "detection_rate_pct": round(detection_rate, 2)
})

overall_total += total
overall_detected += detected
overall_not_detected += not_detected
overall_unreadable += unreadable

# Overall summary
overall_readable = overall_total - overall_unreadable
overall_detection_rate = (overall_detected / overall_readable * 100) if overall_readable > 0 else float('nan')

# For a table-like report
print("\n" + "="*72)
print("CLASS-WISE HAND DETECTION REPORT")
print("="*72)
print(f"{'CLASS':>6} | {'TOTAL':>5} | {'READ':>5} | {'UNREAD':>6} | {'HANDS':>5} | {'NO_HAND':>7} | {'DETECT_%':>8}")
print("-"*72)
for r in report_rows:
    print(f"{r['class']:>6} | {r['total_images']:5d} | {r['readable_images']:5d} | {r['unreadable_files']:6d} | {r['images_with_hands']:5d} | {r['images_with_no_hands']:7d} | {str(r['detection_rate_pct']).rjust(8)}")
print("-"*72)
print(f"{'TOTAL':>6} | {overall_total:5d} | {overall_readable:5d} | {overall_unreadable:6d} | {overall_detected:5d} | {overall_not_detected:7d} | round(overall_detection_rate,2)")
print("="*72 + "\n")
```

CLASS-WISE HAND DETECTION REPORT						
CLASS	TOTAL	READ	UNREAD	HANDS	NO_HAND	DETECT_%
0	70	70	0	42	28	60.0
1	70	70	0	66	4	94.29
2	70	70	0	60	10	85.71
3	70	70	0	69	1	98.57
4	70	70	0	70	0	100.0
5	70	70	0	70	0	100.0
6	70	70	0	59	11	84.29
7	70	70	0	67	3	95.71
8	70	70	0	64	6	91.43
9	70	70	0	70	0	100.0
a	70	70	0	27	43	38.57
b	70	70	0	67	3	95.71
c	70	70	0	49	21	70.0
d	70	70	0	69	1	98.57
e	70	70	0	38	32	54.29
f	70	70	0	70	0	100.0
g	70	70	0	64	6	91.43
h	70	70	0	57	13	81.43
i	70	70	0	59	11	84.29
j	70	70	0	29	41	41.43
k	70	70	0	70	0	100.0
l	70	70	0	69	1	98.57
m	70	70	0	17	53	24.29
n	70	70	0	22	48	31.43
o	70	70	0	36	34	51.43
p	70	70	0	61	9	87.14
q	70	70	0	24	46	34.29
r	70	70	0	56	14	80.0
s	70	70	0	17	53	24.29
t	65	65	0	13	52	20.0
u	70	70	0	65	5	92.86
v	70	70	0	63	7	90.0
w	70	70	0	65	5	92.86
x	70	70	0	52	18	74.29
y	70	70	0	45	25	64.29
z	70	70	0	58	12	82.86
TOTAL	2515	2515	0	1899	616	round(overall_detection_rate,2)

Data Pre- Processing

08

```
# CELL 4 - compute median target count for balancing
detected_counts = [filtered_counts[c] for c in classes]
print("Detected counts per class:", detected_counts)

median_target = int(np.median([c for c in detected_counts if c>0])) # avoid zero classes
min_target = min([c for c in detected_counts if c>0])
max_target = max(detected_counts)

print(f"\nMin detected: {min_target}, Median detected: {median_target}, Max detected: {max_target}")
TARGET_PER_CLASS = median_target
print("Using TARGET_PER_CLASS =", TARGET_PER_CLASS)
```

... Detected counts per class: [42, 66, 60, 69, 70, 70, 59, 67, 64, 70, 27, 67, 49, 69, 38, 70, 64, 57, 59, 29, 70, 69, 17, 22, 36, 61, 24, 56, 17, 13, 65, 63, 65, 52, 45, 58]

Min detected: 13, Median detected: 59, Max detected: 70
Using TARGET_PER_CLASS = 59

```
# CELL 5 - Create balanced dataset by copying originals and augmenting to TARGET_PER_CLASS
os.makedirs(BALANCED_DIR, exist_ok=True)
mp_hands = mp.solutions.hands

# We'll augment only the filtered images (which already have hands)
for cls in classes:
    src_cls = os.path.join(FILTERED_DIR, cls)
    dst_cls = os.path.join(BALANCED_DIR, cls)
    os.makedirs(dst_cls, exist_ok=True)

    # list existing (filtered) images
    imgs = sorted([f for f in os.listdir(src_cls) if f.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp'))])
    if len(imgs) == 0:
        print(f"Warning: class {cls} has zero filtered images. Skipping.")
        continue

    # Copy originals first
    for fname in imgs:
        shutil.copy(os.path.join(src_cls, fname), os.path.join(dst_cls, fname))

    current = len(imgs)
    idx = 0

    # Augment until target
    while current < TARGET_PER_CLASS:
        src = os.path.join(src_cls, imgs[idx % len(imgs)])
        img = safe_imread(src)
        if img is None:
            idx += 1
            continue
        img = resize_max_dim(img)
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        aug = random_augment(img_rgb)
        out_name = f"aug_{idx}_{os.path.basename(src)}"
        out_path = os.path.join(dst_cls, out_name)
        # save in BGR for cv2
        cv2.imwrite(out_path, cv2.cvtColor(aug, cv2.COLOR_RGB2BGR))
        current += 1
        idx += 1

    print("Balanced dataset created at:", os.path.abspath(BALANCED_DIR))

    # quick verification
    for cls in classes:
        p = os.path.join(BALANCED_DIR, cls)
        c = len([f for f in os.listdir(p) if f.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp'))])
        if os.path.isdir(p) else 0
        print(f"{cls}: {c}")
```

Data Preparation

DATA IS READY TO BE TRAINED
AND CLASSIFIED FURTHER.

```
*** Balanced dataset created at: /content/balanced_dataset
```

```
0: 59
1: 66
2: 60
3: 69
4: 70
5: 70
6: 59
7: 67
8: 64
9: 70
a: 59
b: 67
c: 59
d: 69
e: 59
f: 70
g: 64
h: 59
i: 59
j: 59
k: 70
l: 69
m: 59
n: 59
o: 59
p: 61
q: 59
r: 59
s: 59
t: 59
u: 65
v: 63
w: 65
x: 59
y: 59
z: 59
```

```
Extracting landmarks for class '0' (59 images)...
Feat 0: 100% [██████████] 59/59 [00:03<00:00, 16.07it/s]
Extracting landmarks for class '1' (66 images)...
Feat 1: 100% [██████████] 66/66 [00:02<00:00, 23.47it/s]
Extracting landmarks for class '2' (60 images)...
Feat 2: 100% [██████████] 60/60 [00:02<00:00, 23.71it/s]
Extracting landmarks for class '3' (69 images)...
Feat 3: 100% [██████████] 69/69 [00:02<00:00, 24.31it/s]
Extracting landmarks for class '4' (70 images)...
Feat 4: 100% [██████████] 70/70 [00:03<00:00, 21.73it/s]
Extracting landmarks for class '5' (70 images)...
Feat 5: 100% [██████████] 70/70 [00:03<00:00, 20.61it/s]
Extracting landmarks for class '6' (59 images)...
Feat 6: 100% [██████████] 59/59 [00:02<00:00, 24.71it/s]
Extracting landmarks for class '7' (67 images)...
Feat 7: 100% [██████████] 67/67 [00:02<00:00, 24.98it/s]
Extracting landmarks for class '8' (64 images)...
Feat 8: 100% [██████████] 64/64 [00:02<00:00, 23.45it/s]
Extracting landmarks for class '9' (70 images)...
Feat 9: 100% [██████████] 70/70 [00:04<00:00, 16.43it/s]
Extracting landmarks for class 'a' (59 images)...
Feat a: 100% [██████████] 59/59 [00:02<00:00, 23.75it/s]
Extracting landmarks for class 'b' (67 images)...
Feat b: 100% [██████████] 67/67 [00:02<00:00, 24.62it/s]
Extracting landmarks for class 'c' (59 images)...
Feat c: 100% [██████████] 59/59 [00:02<00:00, 24.62it/s]
Extracting landmarks for class 'd' (69 images)...
Feat d: 100% [██████████] 69/69 [00:03<00:00, 18.53it/s]
Extracting landmarks for class 'e' (59 images)...
Feat e: 100% [██████████] 59/59 [00:02<00:00, 22.50it/s]
Extracting landmarks for class 'f' (70 images)...
Feat f: 100% [██████████] 70/70 [00:02<00:00, 24.43it/s]
Extracting landmarks for class 'g' (64 images)...
Feat g: 100% [██████████] 64/64 [00:02<00:00, 24.39it/s]
Extracting landmarks for class 'h' (59 images)...
Feat h: 100% [██████████] 59/59 [00:02<00:00, 23.43it/s]
Extracting landmarks for class 'i' (59 images)...
Feat i: 100% [██████████] 59/59 [00:03<00:00, 17.44it/s]
Extracting landmarks for class 'j' (59 images)...
Feat j: 100% [██████████] 59/59 [00:02<00:00, 25.75it/s]
Extracting landmarks for class 'k' (70 images)...
Feat k: 100% [██████████] 70/70 [00:02<00:00, 25.03it/s]
Extracting landmarks for class 'l' (69 images)...
Feat l: 100% [██████████] 69/69 [00:02<00:00, 25.22it/s]
Extracting landmarks for class 'm' (59 images)...
Feat m: 100% [██████████] 59/59 [00:03<00:00, 19.16it/s]
Extracting landmarks for class 'n' (59 images)...
Feat n: 100% [██████████] 59/59 [00:02<00:00, 21.82it/s]
Extracting landmarks for class 'o' (59 images)...
Feat o: 100% [██████████] 59/59 [00:02<00:00, 25.63it/s]
Extracting landmarks for class 'p' (61 images)...
Feat p: 100% [██████████] 61/61 [00:03<00:00, 16.66it/s]
Extracting landmarks for class 'q' (59 images)...
Feat q: 100% [██████████] 59/59 [00:07<00:00, 7.95it/s]
Extracting landmarks for class 'r' (59 images)...
Feat r: 100% [██████████] 59/59 [00:06<00:00, 9.51it/s]
Extracting landmarks for class 's' (59 images)...
Feat s: 100% [██████████] 59/59 [00:03<00:00, 17.71it/s]
```

```
mp_hands = mp.solutions.hands
dataset_feats = []
dataset_labels = []
class_list = [] # index->folder name mapping

classes_balanced = sorted([d for d in os.listdir(BALANCED_DIR) if os.path.isdir(os.path.join(BALANCED_DIR, d))])
class_list = classes_balanced.copy()

with mp_hands.Hands(static_image_mode=True, min_detection_confidence=MIN_CONFIDENCE) as hands:
    for cls_idx, cls in enumerate(classes_balanced):
        cls_dir = os.path.join(BALANCED_DIR, cls)
        files = sorted([f for f in os.listdir(cls_dir) if f.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp'))])
        print(f"Extracting landmarks for class '{cls}' ({len(files)} images)...")
        for fname in tqdm(files, desc=f"Feat {cls}"):
            p = os.path.join(cls_dir, fname)
            img = safe_imread(p)
            if img is None:
                continue
            img = resize_max_dim(img)
            img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            res = hands.process(img_rgb)
            if not res.multi_hand_landmarks:
                # This should be rare (we filtered earlier) - skip if so
                continue
            lm = res.multi_hand_landmarks[0]
            feats = extract_features_from_landmarks(lm)
            dataset_feats.append(feats)
            dataset_labels.append(cls_idx)

dataset_feats = np.array(dataset_feats, dtype=np.float32)
dataset_labels = np.array(dataset_labels, dtype=np.int64)
print("Features shape:", dataset_feats.shape)
print("Labels shape:", dataset_labels.shape)

# Save to pickle
with open(DATA_PKL_PATH, "wb") as f:
    pickle.dump({'data': dataset_feats, 'labels': dataset_labels, 'classes': class_list}, f)

print("Saved features+labels to:", DATA_PKL_PATH)
```

09

Model Training & Evaluation

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load data (safety)
with open(DATA_PKL_PATH, "rb") as f:
    dd = pickle.load(f)
X = dd['data']
y = dd['labels']
classes = dd['classes']
print("Number of classes:", len(classes))

# Train/test split (stratified)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=RANDOM_SEED, stratify=y)

clf = RandomForestClassifier(n_estimators=200, random_state=RANDOM_SEED, n_jobs=-1)
print("Training RandomForest...")
clf.fit(X_train, y_train)

# Evaluate
y_pred = clf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"Test accuracy: {acc*100:.2f}%\n")
print("Classification report:")
print(classification_report(y_test, y_pred, target_names=classes, digits=4))

# Save model with classes mapping
model_dict = {'model': clf, 'classes': classes}
with open(MODEL_PKL_PATH, "wb") as f:
    pickle.dump(model_dict, f)
print("Saved trained model to:", MODEL_PKL_PATH)
```

Number of classes: 36
Training RandomForest...
Test accuracy: 92.45%

Classification report:				
	precision	recall	f1-score	support
0	0.7273	0.7273	0.7273	11
1	1.0000	1.0000	1.0000	13
2	0.8462	0.9167	0.8800	12
3	0.9333	1.0000	0.9655	14
4	0.8571	0.8571	0.8571	14
5	0.8571	0.8571	0.8571	14
6	1.0000	0.9167	0.9565	12
7	1.0000	1.0000	1.0000	13
8	1.0000	0.8462	0.9167	13
9	1.0000	1.0000	1.0000	14
a	0.9091	0.9091	0.9091	11
b	0.9286	1.0000	0.9630	13
c	0.9231	1.0000	0.9600	12
d	1.0000	1.0000	1.0000	14
e	0.8462	1.0000	0.9167	11
f	1.0000	1.0000	1.0000	14
g	0.9286	1.0000	0.9630	13
h	1.0000	1.0000	1.0000	12
i	1.0000	1.0000	1.0000	12
j	0.9000	0.9000	0.9000	10
k	0.9333	1.0000	0.9655	14
l	0.9231	0.8571	0.8889	14
m	0.8889	0.8000	0.8421	10
n	0.9091	0.9091	0.9091	11
o	0.8333	0.5556	0.6667	9
p	1.0000	1.0000	1.0000	12
q	0.6875	1.0000	0.8148	11
r	1.0000	1.0000	1.0000	12
s	0.8750	0.7778	0.8235	9
t	1.0000	0.8889	0.9412	9
u	1.0000	1.0000	1.0000	13
v	0.9167	0.8462	0.8800	13
w	0.9167	0.8462	0.8800	13
x	0.9167	0.9167	0.9167	12
y	0.8182	0.8182	0.8182	11
z	1.0000	0.9167	0.9565	12
accuracy			0.9245	437
macro avg	0.9243	0.9184	0.9188	437
weighted avg	0.9277	0.9245	0.9238	437

Saved trained model to: model.p

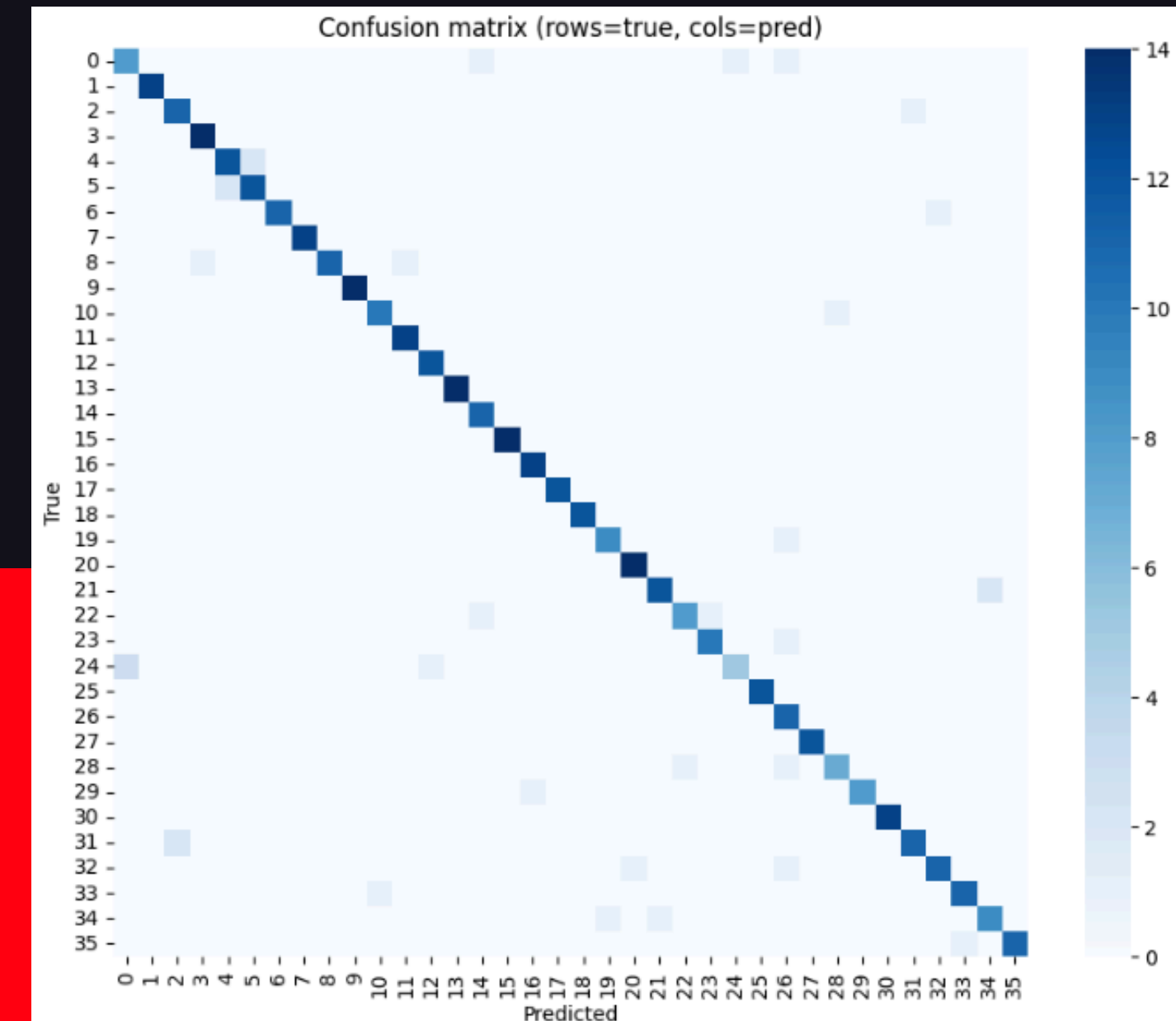
Model Training & Evaluation

```
# CELL 8 - Confusion matrix & class counts
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# compute confusion matrix on X_test
y_pred = clf.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=False, fmt='d', cmap='Blues')
plt.title("Confusion matrix (rows=true, cols=pred)")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()

# class counts
unique, counts = np.unique(dataset_labels, return_counts=True)
for idx, cnt in zip(unique, counts):
    print(f"{classes[idx]}: {cnt} samples")
```





**THANK
YOU!**